Shor's Algorithm Investigation

Introduction

A large portion of encryption on the internet across banking and messaging uses an encryption method called RSA. This works under the assumption that factoring large numbers into just 2 prime factors is relatively impossible, or just takes too long to be feasible. Developed in 1994 by Peter Shor, Shor's algorithm breaks RSA using a large enough quantum computer. It draws on my own implementation of Shor's algorithm in Python and Qiskit, and considers the gap between small demonstrations and a real attack, and the move towards post-quantum cryptography.

How Does RSA Work

To understand why Shor's algorithm works we need to understand RSA. The algorithm works by using a pair of commutative keys, where one key is used to encrypt and the other to decrypt.

Every user has 2 keys, a public key and a private key. To generate keys, two large prime numbers P and Q are multiplied together to give N. The totient is also calculated using $\phi = (p - 1)(q - 1)$. Using this we can choose a public key, E, which must follow 3 rules:

1. It must be coprime to ϕ
2. It must be less that the totient
3. It must not be a factor of totient

Selecting the private key, D, involves satisfying one formula $D*E \bmod(\phi) = 1$

Then to encrypt use the formula: $Message^E \bmod(PQ) = Cipher$
And to decrypt: $Cipher^D \bmod(PQ) = Message$

The security of RSA rests on the fact that N and E are public, but calculating d requires ϕ, and calculating ϕ requires the prime factors P and Q. Multiplying two large primes together is easy, but no efficient classical algorithm is known for factoring N back into P and Q when N is greater than 2048 bits. Shor's algorithm undermines this assumption.

How Shor's Algorithm Works

Currently, classical computers give one output for each input. With superposition, quantum computers can calculate many outputs at once, however, only one random output is returned when measured. Shor's algorithm works in a way where all the incorrect outputs interfere with each other destructively.

Shor's algorithm works by guessing a number that might share a factor with N (discussed in RSA) and then refining the guess until it finds a factor.

Given a number N used to encrypt data, its factors are needed to decrypt data. A guess, g, is made. To find the factors of N we use a mathematical fact: for any two numbers that don't share factors, A and B, if you multiply A by itself enough times it will equal a multiple of B, plus 1. So:

$$A^r = M \times B + 1$$

Eg. 7 and 15: $7^4 = 160 \times 15 + 1$

So for N and our guess, g, we know:

$$g^r = M \times N + 1$$

which is the same as:

$$g^r - 1 = M \times N$$

which is the same as:

$$(g^{(r/2)} + 1) \times (g^{(r/2)} - 1) = M \times N$$

These two terms are better guesses. Unless one of them is itself a multiple of N, each shares a factor with N. Eg. for g = 7, N = 15, r = 4: $7^2 + 1 = 50$ and $7^2 - 1 = 48$, and gcd(50, 15) = 5 and gcd(48, 15) = 3.

Quantum computers excel at finding r, compared to classical computers. The quantum computer takes in a superposition of all possible powers, x, and calculates g^x, then calculates how much bigger the result is than a multiple of N (the remainder). Using superposition we can perform many calculations at once, this is where quantum computers excel over classical computers.

Measuring the remainder would output a random number, so to make all the incorrect outputs destructively interfere with each other we also need to remember that:

1. The remainder has a repeating property. If $g^x = M \times N + R$, then $g^{(x+r)} = M' \times N + R$, so the remainder is the same (with a different multiple of N).

2. If you input a superposition, and the random output could have come from more than one element of the superposition, the state left is a superposition of just those elements.

So after measuring the remainder, the superposition is left with only the powers x that give that remainder, and these are all r apart. They repeat with a frequency of 1/r. To find the frequency, and so r, we use a quantum Fourier transform. Applying it to our superposition, which repeats with a frequency of 1/r, causes all the frequencies that aren't present to destructively interfere, leaving only 1/r as the output. We then invert this to give r.

And finally, if r is even, we raise our guess to the power r/2 and add or subtract 1. As long as the result isn't an exact multiple of N, we are guaranteed to have numbers that share factors with N. If not, we repeat with a new guess.

Simply use Euclid's algorithm to find a factor of N, then divide N by it to find the other, and break the encryption.

References

Very helpful videos –

How Quantum Computers Break Encryption | Shor's Algorithm Explained

RSA Algorithm - How does it work? - I'll PROVE it with an Example! -- Cryptography - Practical TLS

Helpful YouTube Channel –

Sandro Mareco

Helpful GitHub –

https://github.com/mett29/Shor-s-Algorithm/blob/master/Shor.ipynb